

Assignment 02 – Untyped Lambda Calculus

Ryan Snell

Question 1 — Term Forms

For each term, the outermost syntactic form is identified below.

Term	Outermost Form
x	Variable
$\lambda x. x$	Abstraction
$(\lambda x. x) y$	Application
$\lambda x. (x y)$	Abstraction

Question 2 — Free Variables

Term: $\lambda x. (x y)$

The abstraction binds x , so the occurrence of x in the body is bound. The occurrence of y is not bound by any enclosing abstraction.

Free variables: $\{y\}$

Question 3 — More Free Variables

Term: $\lambda x. \lambda y. (x z)$

Both x and y are bound by their respective abstractions. The variable y is bound but never used in the body. The variable z is not bound by anything.

Free variables: $\{z\}$

Question 4 — Scope and Shadowing

Term: $\lambda x. \lambda x. x$

The inner abstraction $\lambda x. x$ re-binds the name x , shadowing the outer λx for the remainder of its scope. Because the final x occurs inside the body of the inner abstraction, it refers to the closest (innermost) binder.

Answer: The second (innermost) λx binds the final x , because variable references always resolve to the nearest enclosing binder with the same name.

Question 5 — Alpha-Equivalence

Term A: $\lambda x. \lambda y. x$

Term B: $\lambda a.\lambda b.a$

Both terms have the same structure: an outer abstraction whose body is another abstraction, whose body is a reference to the variable bound by the outer abstraction. Renaming the bound names x to a and y to b throughout (consistently, with no capture) turns Term A into Term B.

Answer: Yes, they are alpha-equivalent — they differ only in the names chosen for the bound variables, and the binding structure is identical.

Question 6 — Not Alpha-Equivalent

Term A: $\lambda x.\lambda y.x$

Term B: $\lambda x.\lambda y.y$

Alpha-equivalence only allows renaming of bound variables; it can never change which binder a variable occurrence refers to. In Term A, the body x refers to the outer abstraction's parameter. In Term B, the body y refers to the inner abstraction's parameter. No consistent renaming of bound variables can turn one binding pattern into the other.

Answer: No. Even though both terms take two arguments, Term A always returns the first argument while Term B always returns the second argument — a genuine difference in behavior/structure, not just a naming difference.

Question 7 — OCaml AST

Term: $\lambda x.(x\ y)$

Using the given term type:

```
Abs ("x", App (Var "x", Var "y"))
```

Question 8 — OCaml AST with Nested Functions

Term: $\lambda x.\lambda y.x$

```
Abs ("x", Abs ("y", Var "x"))
```

Question 9 — Simple Beta-Reduction

Term: $(\lambda x.x)(\lambda y.y)$

This is a redex with parameter x , body x , and argument $\lambda y.y$. Beta-reduction substitutes the argument for every free occurrence of the parameter in the body.

Substitution step: $[x \mapsto \lambda y.y] x = \lambda y.y$

Result: $\lambda y.y$

Question 10 — Beta-Reduction with a Nested Function

Term: $(\lambda x.\lambda y.x)(\lambda z.z)$

The redex substitutes $\lambda z.z$ for x in the body $\lambda y.x$. Since the bound variable of the body's abstraction is y (different from x), there is no capture, and the substitution passes straight through into the inner body.

Substitution step: $[x \mapsto \lambda z.z] (\lambda y.x) = \lambda y.(\lambda z.z)$

Result: $\lambda y.\lambda z.z$

Question 11 — Multi-Step Reduction

Term: $(\lambda x.x)((\lambda y.y)(\lambda z.z))$

Reducing the outermost redex first (normal order):

$$\begin{array}{ll} (\lambda x.x) ((\lambda y.y)(\lambda z.z)) & \\ \rightarrow (\lambda y.y)(\lambda z.z) & [x \mapsto (\lambda y.y)(\lambda z.z)] \\ \rightarrow \lambda z.z & [y \mapsto \lambda z.z] \end{array}$$

Final result: $\lambda z.z$

Question 12 — Basic Substitution

Expression: $[x \mapsto y] (x z)$

Substitution distributes over application, replacing every free x with y . z is untouched since it is not the variable being substituted.

Result: $y z$

Question 13 — Bound Variables and Substitution

Expression: $[x \mapsto y] (\lambda x.x)$

The outer substitution targets free occurrences of x . Inside $\lambda x.x$, the abstraction re-binds the name x , so the x in the body refers to this new binder, not to the x being substituted for — it is a bound occurrence, not a free one. Substitution only ever replaces free occurrences.

Result: $\lambda x.x$ (unchanged), because the x inside the abstraction is bound, and substitution never touches bound occurrences of the variable it is renaming.

Question 14 — Variable Capture

Expression: $[x \mapsto y] (\lambda y.x)$

1. Naïve text replacement

Naïvely swapping every textual x for y gives: $\lambda y.y$

2. Why this is incorrect

The original term $\lambda y.x$ is a function that ignores its argument and always returns the free variable x from the enclosing context. After naïve substitution, $\lambda y.y$ is the identity function — it now returns whatever argument it is given. The free variable y that we substituted in was accidentally **captured** by the λy binder, completely changing the meaning of the term.

3. Correct substitution via alpha-renaming

First alpha-rename the bound y to a fresh name, e.g. z , which does not occur free in the substituted value:

$\lambda y.x$ is alpha-equivalent to $\lambda z.x$ (bound variable renamed)

Now substitute safely, since z cannot capture y :

$[x \mapsto y] (\lambda z.x) = \lambda z.y$

Correct result: $\lambda z.y$

Question 15 — Small OCaml Programming Exercise

A. Count AST Nodes

Each constructor (Var, Abs, App) contributes exactly one node, plus the nodes of its sub-term(s).

```
let rec size (t : term) : int =
  match t with
  | Var _ -> 1
  | Abs (_, body) -> 1 + size body
  | App (t1, t2) -> 1 + size t1 + size t2

(* size (Abs ("x", Var "x")) = 1 + size (Var "x") = 1 + 1 = 2 *)
```

B. Determine Whether a Term Is a Value

For this assignment, only abstractions are considered values (they are the only terms that a leftmost-outermost or call-by-value evaluator would stop reducing).

```
let is_value (t : term) : bool =
  match t with
  | Abs (_, _) -> true
  | _ -> false

(* is_value (Abs ("x", Var "x")) = true *)
(* is_value (Var "x") = false *)
```

Full working versions of these functions, together with the Question 7 and Question 8 translations, are included in the accompanying OCaml project (ocaml/lib/lambda.ml), with a small demo runner in ocaml/bin/main.ml.